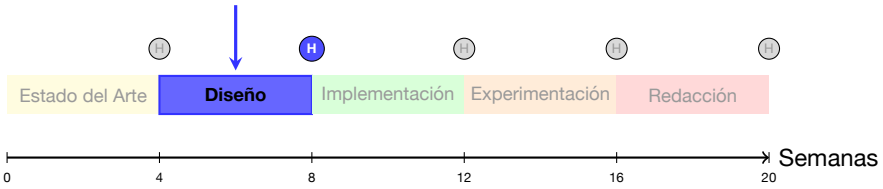
Documento hito #4 (de 6) de mes 2



Arquitectura de Paralelización con Apache Spark RDDs en un Algoritmo Genético para Selección Automática de Características

TFM: Selección Automática de Características en Algoritmos de Clasificación Mediante Algoritmos Genéticos y Métodos de Envoltorio usando Apache Spark RDDs

Entorno de ejecución para TFM: MacBook Pro M5 · Spark `local[8]` · Python + PySpark

1. Fundamentos de Apache Spark

Apache Spark es un motor de computación distribuida diseñado para el procesamiento paralelo de datos a gran escala (Zaharia et al., 2016). A diferencia de otros *frameworks* de procesamiento por lotes, Spark mantiene los datos intermedios en memoria, lo que reduce drásticamente la latencia en algoritmos iterativos como los que se emplean en este trabajo. A continuación se describen sus cinco conceptos esenciales.

1.1 RDD (*Resilient Distributed Dataset*)

El RDD constituye la abstracción fundamental de Spark (Apache Spark Documentation, 2024). Se trata de una colección de elementos distribuida, inmutable y tolerante a fallos. Sus propiedades clave son:

- *Distribución*: los datos se dividen en particiones que se reparten entre los nodos del clúster (o, en modo local, entre los *threads* disponibles). Cada partición puede procesarse de forma independiente.
- *Inmutabilidad*: un RDD no se modifica *in-place*; toda operación sobre él genera un nuevo RDD, lo que simplifica la gestión de concurrencia.
- *Resiliencia*: Spark registra el linaje (*lineage*) de cada RDD, es decir, la secuencia de transformaciones que lo originó. Si una partición se pierde por un fallo, Spark la reconstruye automáticamente reejecutando dichas transformaciones desde los datos de origen, sin requerir replicación explícita.

1.2 Driver

El Driver es el proceso principal que ejecuta el programa del usuario. Sus responsabilidades son:

1. Crear el SparkContext (`sc`), punto de entrada a la funcionalidad de Spark.
2. Definir las transformaciones y acciones sobre los RDDs.
3. Construir el DAG (*Directed Acyclic Graph*) de ejecución a partir de dichas operaciones.
4. Coordinar la distribución de tareas a los Workers y recopilar los resultados.

El Driver no ejecuta cómputo sobre los datos distribuidos; su rol es exclusivamente de orquestación.

1.3 Worker

Los *Workers* son los procesos (o *threads*, en modo local) que realizan el cómputo efectivo. Cada Worker recibe una o más particiones del RDD junto con la función serializada que debe aplicar, ejecuta el trabajo asignado y devuelve los resultados al Driver. En un despliegue en clúster, los Workers residen en nodos físicos distintos; en modo `local[k]`, Spark simula k Workers como *threads* de la JVM local.

1.4 Transformation

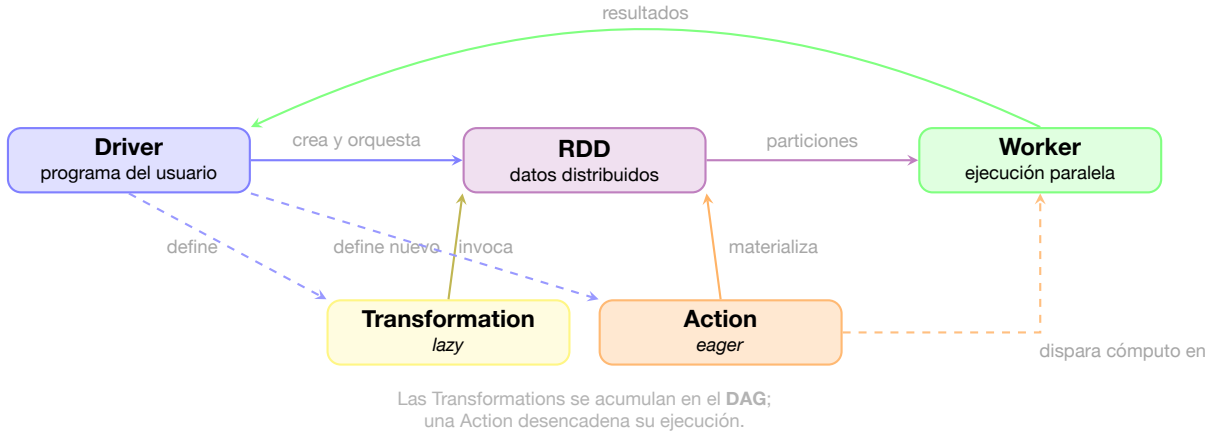
Una Transformation es una operación *lazy* que define una nueva computación sobre un RDD sin ejecutarla. Spark se limita a registrar la operación en el DAG. Ejemplos habituales son `map()`, `filter()`, `flatMap()` y `parallelize()`. Las Transformations permiten a Spark optimizar el plan de ejecución completo antes de materializar ningún resultado.

1.5 Action

Una Action es una operación *eager* que fuerza la ejecución de todas las Transformations acumuladas en el DAG. En ese momento, Spark planifica las tareas, las distribuye a los Workers y materializa los resultados, devolviéndolos al Driver o escribiéndolos en un sistema de almacenamiento. Ejemplos típicos son `collect()`, `count()`, `take()` y `reduce()`.

1.6 Relación entre conceptos

El siguiente diagrama resume la arquitectura de Spark y las relaciones entre sus componentes principales.



En síntesis, el Driver construye un plan de ejecución mediante Transformations sobre RDDs; cuando una Action lo requiere, Spark distribuye el trabajo entre los Workers, que procesan las particiones en paralelo y devuelven los resultados al Driver. Esta arquitectura *lazy* permite a Spark optimizar el plan completo antes de ejecutar ningún cómputo, minimizando transferencias de datos innecesarias.

2. Aplicación de Spark al Algoritmo Genético

Una vez descritos los fundamentos de Spark, esta sección concreta su uso en el marco del Algoritmo Genético (GA) para selección automática de características.

2.1 Paralelización de la evaluación de fitness

En cada generación del GA se dispone de una población de N individuos, cada uno representado como un cromosoma binario **c** de longitud igual al número de características del dataset:

$$\mathbf{c} = [c_1, c_2, \dots, c_D], \quad c_j \in \{0, 1\}$$

donde D es el número de características del dataset estudiado.

La evaluación de cada individuo implica entrenar el clasificador con el subconjunto de *features* definido por su cromosoma y calcular la métrica de *fitness* (e.g., *accuracy* en validación cruzada más penalización

por número de características). Esta es la operación computacionalmente más costosa del proceso. Dado que la evaluación de *fitness* es independiente entre individuos, resulta directamente paralelizable.

Spark explota esta independencia asignando cada evaluación a un Worker distinto. Con la configuración `local[8]`, se evalúan 8 individuos simultáneamente, procesando la población completa en aproximadamente $\lceil N/8 \rceil$ rondas de ejecución paralela.

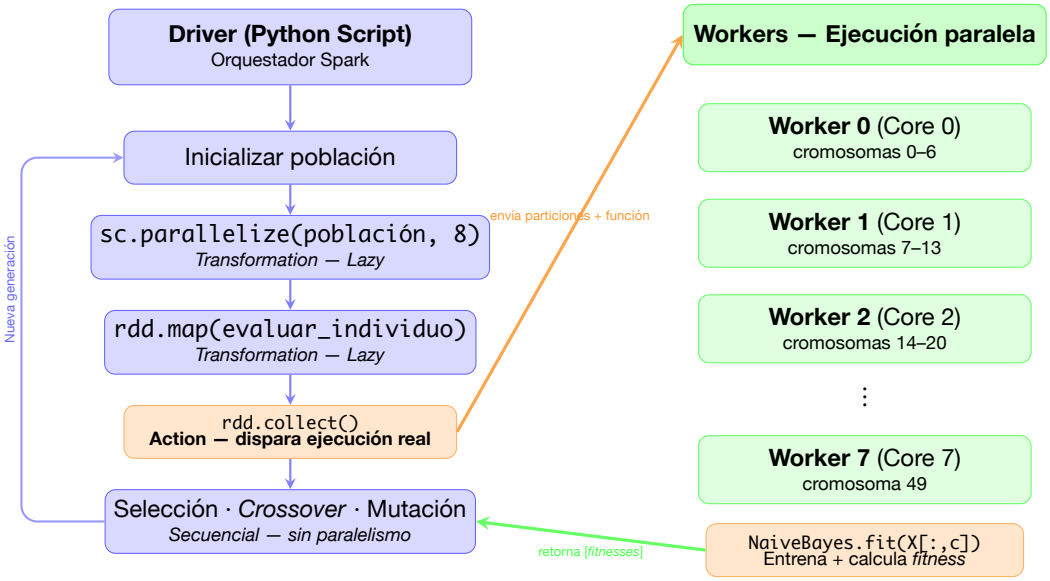
Las operaciones genéticas, en cambio, se ejecutan de forma secuencial en el Driver, al presentar dependencias lógicas entre individuos (el resultado de la selección afecta al *crossover*, y este a la mutación).

Operación	Ejecutada en	¿Paralelizada?	Motivo
Entrenamiento del clasificador	Workers	Sí (8 simultáneamente)	Sin dependencias entre individuos
Cálculo del *fitness*	Workers	Sí (8 simultáneamente)	Evaluación independiente
Selección de padres (torneo)	Driver	No	Requiere comparar *fitnesses* del conjunto
Crossover uniforme	Driver	No	Depende de los padres seleccionados
Mutación	Driver	No	Opera sobre individuos ya cruzados
Elitismo / nueva generación	Driver	No	Gestión global de la población

Nota: La paralelización de las operaciones genéticas es teóricamente posible, pero su coste computacional es despreciable frente a la evaluación, lo que no justifica la complejidad añadida.

2.2 Estructura de RDDs en el GA

En el contexto de este TFM, el RDD representa la población de la generación actual del GA, donde cada elemento es un cromosoma (vector binario). Spark divide esta población en particiones que se distribuyen entre los Workers, permitiendo la evaluación paralela sin comunicación entre nodos durante la ejecución.



2.3 Roles de Driver y Workers en el GA

En el GA, el Driver gestiona la población, aplica los operadores genéticos (selección, *crossover*, mutación) y coordina la ejecución distribuida. Los Workers (8 *threads* JVM en modo `local[8]`) reciben las particiones del RDD junto con la función de evaluación serializada, entrenan los clasificadores en paralelo y devuelven los valores de *fitness* al Driver.

La población se instancia inicialmente en el Driver como una estructura local (e.g., array NumPy). Para distribuirla, se serializa y paraleliza mediante `sc.parallelize()`, dividiéndola en 8 particiones, una por Worker:

```
# Pseudocódigo — estructura básica del bucle generacional
# Ejemplo con una población de 50 individuos y 30 características cada uno
N = 50
K = 30
población = inicializar_población(n=N, n_features=K)  # Driver

for generación in range(N_GENERACIONES):
    rdd_población   = sc.parallelize(población, numPartitions=8)  # Transformation
    rdd_evaluado    = rdd_población.map(evaluar_individuo)         # Transformation
    fitnesses       = rdd_evaluado.collect()                       # Action → ejecución real

    población = operadores_genéticos(población, fitnesses)        # Driver (secuencial)
```

Cada generación del GA reconstruye el RDD a partir de la nueva población modificada por los operadores genéticos. Aunque esto implica serialización repetida, el coste es despreciable frente al entrenamiento de los clasificadores en los *workers* [típicamente <1% del tiempo total de generación].

2.4 Distribución física de una población ejemplo (N=50 cromosomas en 8 particiones)

Spark reparte los individuos de forma aproximadamente uniforme (`floor(50/8) = 6`, con 2 particiones de 7 elementos):

```
┌──────────────────────────────────────────────────────────────┐
│                  RDD: rdd_población (50 cromosomas)          │
├──────────────────────────┬───────────────────────────────────┤
│  Partición 0 → Core 0    │  Partición 4 → Core 4             │
│  cromosomas 0 – 6        │  cromosomas 28 – 34               │
├──────────────────────────┼───────────────────────────────────┤
│  Partición 1 → Core 1    │  Partición 5 → Core 5             │
│  cromosomas 7 – 13       │  cromosomas 35 – 41               │
├──────────────────────────┼───────────────────────────────────┤
│  Partición 2 → Core 2    │  Partición 6 → Core 6             │
│  cromosomas 14 – 20      │  cromosomas 42 – 48               │
├──────────────────────────┼───────────────────────────────────┤
│  Partición 3 → Core 3    │  Partición 7 → Core 7             │
│  cromosomas 21 – 27      │  cromosoma  49                    │
└──────────────────────────┴───────────────────────────────────┘
```

Cada partición reside en la RAM local del *worker* asignado. Durante la fase de evaluación, cada *worker* opera exclusivamente sobre sus cromosomas, sin comunicación con los demás [patrón *embarrassingly parallel*].

2.5 Ejecución del clasificador en los Workers

El clasificador se entrena exclusivamente en los Workers, nunca en el Driver. La función `evaluar_individuo` —que recibe un cromosoma, filtra las columnas del dataset, entrena el clasificador y

retorna el *fitness*— se serializa y envía a cada *worker* como una *closure* de Python. La ejecución remota es forzada por el patrón `map()` + `collect()`.

En las primeras iteraciones del TFM se empleará Naive Bayes (Gaussian o Multinomial, según la naturaleza de los *features* seleccionados) como clasificador base dentro del método *wrapper*. Su bajo coste de entrenamiento —proporcional a $O(n \cdot d)$, donde n es el número de instancias y d el número de *features* del subconjunto activo— lo hace especialmente adecuado para evaluar con rapidez las numerosas combinaciones que el GA explora generación tras generación. Esta elección permite validar la arquitectura de paralelización antes de sustituirlo por clasificadores más complejos en fases posteriores.

Siguiendo la semántica *lazy* descrita en la Sección 1, la ejecución no ocurre en el momento de definir el `map()`, sino cuando se invoca `collect()`. En ese instante, Spark materializa el plan de ejecución y cada Worker lleva a cabo el entrenamiento. El clasificador tiene acceso a los datos de entrenamiento, que se difunden a los Workers mediante *broadcast variables* para evitar transferencias repetidas en cada generación.

2.6 Ejemplo de *timeline* de ejecución por generación

```
t =   0 ms  | Driver invoca collect() → Spark compila el DAG y envía tareas
t =   5 ms  | Spark serializa la función y el cromosoma; los Workers reciben particiones
t =   5 ms  | ┌─ Worker 0: NaiveBayes.fit(X[:, cromosoma_0]) → fitness = 0.924
            | ├─ Worker 1: NaiveBayes.fit(X[:, cromosoma_7]) → fitness = 0.881
            | ├─ Worker 2: NaiveBayes.fit(X[:, cromosoma_14])→ fitness = 0.956
            | ├─ ...  (Workers 3–7 en paralelo)
t = 490 ms  | └─ Worker 7: NaiveBayes.fit(X[:, cromosoma_49])→ fitness = 0.903
t = 495 ms  | Workers envían resultados (lista de fitnesses) al Driver
t = 497 ms  | Driver recibe [0.924, 0.881, 0.956, ..., 0.903] → operadores genéticos
```

Nota: Naive Bayes en scikit-learn es un algoritmo de coste reducido que no requiere aceleración GPU. En el chip Apple M5, su ejecución aprovecha las instrucciones vectoriales SIMD de los P-cores, resultando en tiempos de entrenamiento muy bajos por individuo, lo que maximiza el beneficio de la paralelización frente a clasificadores más pesados.

2.7 Transformations y Actions en el bucle generacional

Los conceptos de Transformation y Action introducidos en la Sección 1 se manifiestan de forma concreta en cada generación del GA:

```
Paso 1: sc.parallelize(población, 8)
        → TRANSFORMATION (Lazy): Spark anota "dividir en 8 particiones"
        → [NO] Ningún dato se mueve aún

Paso 2: rdd.map(evaluar_individuo)
        → TRANSFORMATION (Lazy): Spark anota "aplicar función a cada elemento"
        → [NO] Ningún clasificador se entrena aún

Paso 3: rdd.collect()
        → ACTION (Eager): ¡Spark ejecuta todo el plan!
     → [SI] Datos se distribuyen, clasificadores se entrenan, fitnesses retornan al Driver
```

Operación Spark	Tipo	Ejecución	Rol en el GA
`sc.parallelize (población, 8)`	Transformation	Lazy	Distribuye los N cromosomas
`rdd.map (evaluar_individuo)`	Transformation	Lazy	Define la evaluación *wrapper*

Operación Spark	Tipo	Ejecución	Rol en el GA
`rdd.collect()`	Action	Eager	Punto de ejecución real
`rdd.take(n)`	Action	Eager	Alternativa para depuración parcial

`collect()` constituye el cuello de botella de cada generación: su tiempo de ejecución domina completamente el coste total, ya que los operadores genéticos en el Driver consumen menos del 5% del tiempo generacional. En el caso de Naive Bayes, este cuello de botella es notablemente menor que con clasificadores más complejos, lo que acorta el tiempo total del GA y facilita la exploración de un mayor número de generaciones en el mismo tiempo de cómputo. La optimización futura puede pasar por ajustar el número de particiones o sustituir el clasificador base por alternativas con mayor capacidad discriminativa.

Referencias

- Apache Spark Documentation. (2024). *RDD Programming Guide*. https://spark.apache.org/docs/latest/rdd-programming-guide.html
- Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., … & Stoica, I. (2016). Apache Spark: a unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65.